Homework 5 - Writeup

Team members

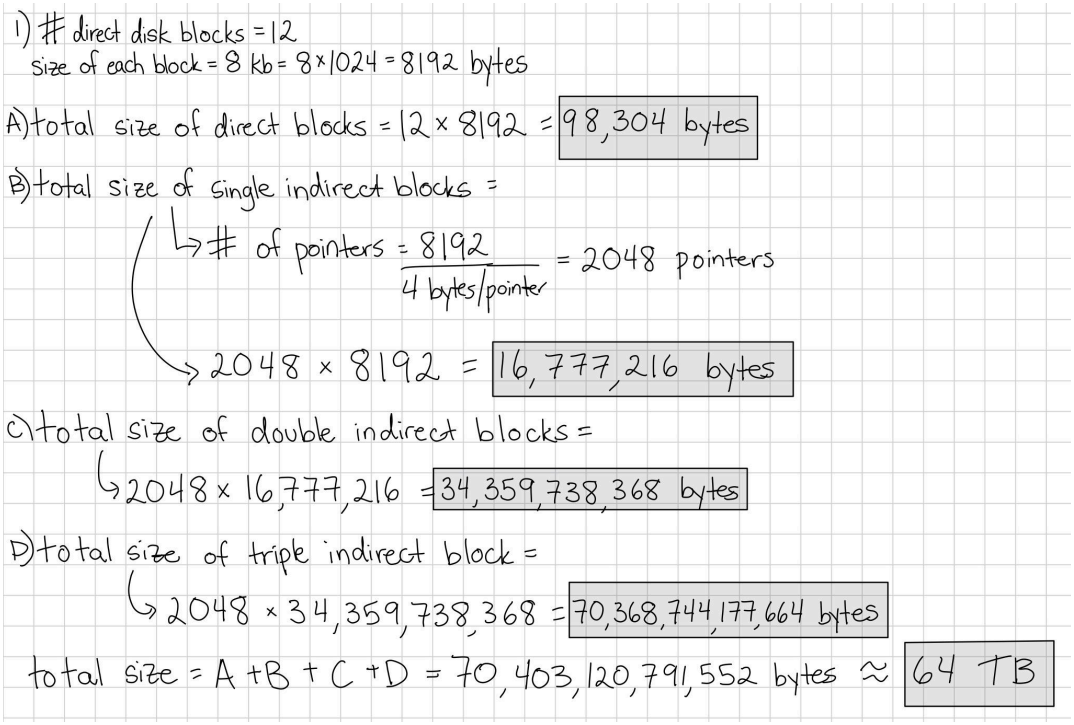
Tania Ortiz-Rosales

Soniya Pradeepkumar Phaltane

1. Consider a file system that uses inodes to represent files. Disk blocks are 8 KB in size, and a pointer to a disk block requires 4 bytes. This file system has 12 direct disk blocks, as well as single, double, and triple indirect disk blocks. What is the maximum size of a file that can be stored in this file system?

   Answer:

   - 64 TB
   - Computation:



2. Briefly explain why logging metadata updates ensures recovery of a file system after a filesystem crash.

   Answer:

   - Logging the metadata updates ensures recovery of a file system after a file system crash because the metadata essentially acts as a journal. For

example, the metadata allows crash recovery. That means that the system can replay the log once a reboot of the system occurs. By doing so, it reapplies the already completed transactions and discards any of the incomplete ones thus restoring consistency within the entire system. The logging also minimizes corruption as well. For example, if a crash were to occur in the middle of an update, it would put the entire file system in an inconsistent state. Therefore, the journaling would also prevent this from happening by ensuring the transactions are fully completed before they are committed. Finally, there is a fast recovery time after a crash as well once the log is replayed and thus the overall downtime is reduced as well.

3. Fragmentation on a storage device can be eliminated through compaction. Typical disk devices do not have relocation or base registers (such as those used when memory is to be compacted), so how can we relocate files? Give three reasons why compacting and relocating files are often avoided.

   Answer:

   - There are multiple reasons why compacting and relocating files are often avoided. For example, the first reason would be the overall performance overhead. To elaborate, disk compaction overall is an extremely slow and also a resource intensive task, especially when there are larger disks at play. So, a large amount of read and write operations would increase the overall wear on the hardware and slow down the entire system during the process. The second reason would be that modern file systems reduce the need of compacting and relocating files. For example, current journal file systems and designs help mitigate fragmentation naturally. Finally, the compacting and relocating files also put the files at a high risk of data loss or corruption. For example, if a power failure or a crash were to occur during these actions then it would leave the files in an inconsistent state. Therefore, the metadata updates would have to be perfectly synchronized in order to avoid filesystem corruption.

4. Consider a disk where changing the direction of the seek incurs an additional delay of 3 msecs. Give a disk scheduling policy that this delay harms the most.

Answer:

- A disk scheduling policy where this delay would cause the most harm is Shortest Seek Time First. It would harm this policy the most because SSTF always selects the request that is closest to the current disk head position in order to minimize seek time. However, this change would cause constant direction changes therefore increasing the seek time. Beyond the increased overall seek time, it would also cause a higher latency for requests compared to any other policy and also there is a reduced overall throughput in the policy as the delay would slow down the process time as well.

5. Given a mounted file system with write operations underway, and a system crash or power loss, what must be done before the file system is remounted if: (a) The file system is not log structured? (b) The file system is log-structured?

   Answer:

   - (A) - If the file system is not log structure, this would leave the data in an inconsistent state. Therefore, before the file system is remounted, a file system check must be performed. This will scan the inodes, directory structure, and free block lists to detect and fix any corruption found within the system. Once this is identified, the orphaned inodes, incorrect files fixes, and missing directory entries have to all be fixed. Finally, if it's possible, there must also be a recovery of either lost or partially written files by moving them to a secondary lost and found directory. However, overall, there is no guarantee that there will be a recovery of all lost data.
   - (B) - If the file system is log structured, then the recovery is a lot quicker. Before the file system is remounted, the log must be replayed first which ensures that all updates were written and therefore, applies only the completed transactions to ensure data consistency. Secondly, it would discard any and all uncompleted transactions since they were not committed into the log. Finally, since the metadata would now be consistent due to the logging, the

overall remounting time would be a lot quicker than a non-log structured system.